

FinShare

Group Event Checklist — Complete Reference

Volunteer Model + Basic / Advanced Mode + Audit Logging

For Abhilash • April 2026 • Supersedes all previous checklist documents

SCOPE — This is the single authoritative reference for the FinShare checklist feature. It supersedes FinShare_EventPhase1_Design.docx (checklist sections), FinShare_ChecklistVolunteerModel.docx, FinShare_ChecklistV3_BasicAdvanced.docx, and the previous version of FinShare_Checklist_Complete.docx. Discard all previous checklist documents. Use only this one.

DESIGN VALIDATED — Design reviewed and confirmed ready for implementation. Volunteer claim model ✓ | Quantity-based claiming ✓ | Basic & Advanced modes ✓ | DB structure clean & scalable ✓ | Status automation via SP ✓ | Audit logging reuses existing infrastructure ✓ | API paths lowercase resource-based ✓ | ApiResponse<T> wrapper on all endpoints ✓

Section 1 — What Changed from Phase 1 Design

The original Phase 1 design used a forced-assignment model: each checklist item had a single AssignedMemberId and one person was responsible. Real-world usage (BBQ groups, trip planning) shows that organizers post open lists and members voluntarily claim items. The design has been updated to reflect this, and further refined to support Advanced mode and checklist audit logging.

Area	Phase 1 (original)	This Design (final)
Item ownership	Single AssignedMemberId (forced)	EventChecklistClaims table — many members claim voluntarily
Claiming	Organizer assigns	Members self-claim ("I'll take this")
Quantity	No quantity concept	QuantityNeeded + QuantityClaimed per member
Mandatory items	Not supported	IsMandatory flag (optional, Advanced mode)
Assignment	Always required	AssignedMemberId nullable — Advanced mode only
Status	Not defined	Open Partially Taken Fully Taken Completed
Suggested member	Not supported	SuggestedMemberId (non-enforced hint)
Withdraw claim	Not supported	DELETE .../claim endpoint
Audit logging	Not supported	Uses existing WriteAuditLog — no new table

Section 2 — Database Changes

2.1 ALTER dbo.EventChecklistItems

Apply these changes to the table created in Phase 1. Run in order.

```
-- 1. Remove the old forced-assignment column (Phase 1 design)
ALTER TABLE dbo.EventChecklistItems
    DROP COLUMN AssignedMemberId;

-- 2. Volunteer Model additions
ALTER TABLE dbo.EventChecklistItems
    ADD QuantityNeeded      INT          NOT NULL DEFAULT 1,
        QuantityUnit        NVARCHAR(50) NULL,
        SuggestedMemberId   BIGINT       NULL
        REFERENCES dbo.Members (MemberId);

-- 3. Basic vs Advanced mode flags
ALTER TABLE dbo.EventChecklistItems
    ADD IsMandatory        BIT          NOT NULL DEFAULT 0,
        AssignedMemberId   BIGINT       NULL
        REFERENCES dbo.Members (MemberId);
-- NULL = Basic (open). Value = Advanced (assigned to member).
```

2.2 Final Shape — dbo.EventChecklistItems

```
dbo.EventChecklistItems
-----
ChecklistItemId  BIGINT IDENTITY(1,1) PRIMARY KEY
EventId          BIGINT          NOT NULL  FK → dbo.GroupEvents (EventId)
ItemName         NVARCHAR(300)  NOT NULL
QuantityNeeded   INT           NOT NULL  DEFAULT 1
QuantityUnit     NVARCHAR(50)  NULL      e.g. "bottles", "packets"
SuggestedMemberId BIGINT       NULL      FK → Members (hint, non-enforced)
IsMandatory      BIT          NOT NULL  DEFAULT 0
AssignedMemberId BIGINT       NULL      FK → Members (Advanced mode)
Status           NVARCHAR(30)  NOT NULL  DEFAULT 'Open'
                                           Open | Partially Taken
                                           | Fully Taken | Completed
CreatedAt        DATETIME2    NOT NULL  DEFAULT GETUTCDATE()
UpdatedAt        DATETIME2    NULL
```

2.3 CREATE dbo.EventChecklistClaims (New Table)

```
CREATE TABLE dbo.EventChecklistClaims
(
    ClaimId          BIGINT          IDENTITY(1,1) PRIMARY KEY,
    ChecklistItemId   BIGINT          NOT NULL
        REFERENCES dbo.EventChecklistItems (ChecklistItemId)
        ON DELETE CASCADE,
    MemberId         BIGINT          NOT NULL
        REFERENCES dbo.Members (MemberId),
    QuantityClaimed   INT             NOT NULL DEFAULT 1,
    ClaimedDate       DATETIME2      NOT NULL DEFAULT GETUTCDATE(),
    IsWithdrawn       BIT             NOT NULL DEFAULT 0,
    WithdrawnDate     DATETIME2      NULL,
    CONSTRAINT UQ_Claim_Item_Member UNIQUE (ChecklistItemId, MemberId)
    -- One claim record per member per item.
    -- Withdrawal: UPDATE IsWithdrawn=1. Re-claim: UPDATE IsWithdrawn=0.
);
```

2.4 Performance Index on EventChecklistClaims

Add this index immediately after the CREATE TABLE. Covers the most frequent query pattern in `sp_ClaimChecklistItem` and `sp_GetEventChecklist`.

```
-- Speeds up: claim over-guard check, aggregate SUM queries, claim list lookups
CREATE INDEX IX_EventChecklistClaims_ItemId_Withdrawn
    ON dbo.EventChecklistClaims (ChecklistItemId, IsWithdrawn)
    INCLUDE (MemberId, QuantityClaimed);

-- Covers: WHERE ChecklistItemId=? AND IsWithdrawn=0
-- INCLUDE avoids key lookup for the columns most needed in SELECT
```

2.5 NO new audit table required

IMPORTANT — Do NOT create a new audit table or stored procedure for checklist logging. The existing AuditLogs table and `sp_WriteAuditLog` are already in place. All checklist audit entries are written using the existing `IAuditLogRepository.WriteAsync()` method. See Section 7 for full implementation details.

2.5 Status Reference

Status	Meaning	When Set
Open	No active claims	Default on creation
Partially Taken	Some quantity claimed, not all	Auto via SP after claim
Fully Taken	All quantity claimed by members	Auto via SP after claim
Completed	Organizer confirmed item was actually brought	PUT by organizer only

Section 3 — Stored Procedures

Five procedures total. Three updated from Phase 1; two new. No new audit-specific stored procedure — the existing `sp_WriteAuditLog` handles all audit writes.

3.1 sp_AddChecklistItem — ALTER

```
ALTER PROCEDURE dbo.sp_AddChecklistItem
    @EventId          BIGINT,
    @ItemName          NVARCHAR(300),
    @QuantityNeeded    INT          = 1,
    @QuantityUnit       NVARCHAR(50) = NULL,
    @SuggestedMemberId BIGINT       = NULL,
    @IsMandatory        BIT          = 0,
    @AssignedMemberId   BIGINT       = NULL
AS BEGIN
    SET NOCOUNT ON;
    INSERT INTO dbo.EventChecklistItems
        (EventId, ItemName, QuantityNeeded, QuantityUnit,
         SuggestedMemberId, IsMandatory, AssignedMemberId, Status)
    VALUES
        (@EventId, @ItemName, @QuantityNeeded, @QuantityUnit,
         @SuggestedMemberId, @IsMandatory, @AssignedMemberId, 'Open');
```

```

        SELECT SCOPE_IDENTITY() AS ChecklistItemId;
    END

```

3.2 sp_UpdateChecklistItem — ALTER

```

ALTER PROCEDURE dbo.sp_UpdateChecklistItem
    @ChecklistItemId    BIGINT,
    @ItemName           NVARCHAR(300) = NULL,
    @QuantityNeeded     INT           = NULL,
    @QuantityUnit       NVARCHAR(50) = NULL,
    @SuggestedMemberId  BIGINT       = NULL,
    @IsMandatory        BIT          = NULL,
    @AssignedMemberId   BIGINT       = NULL,
    @Status             NVARCHAR(30) = NULL
AS BEGIN
    SET NOCOUNT ON;

    -- Completion validation: only allow Completed if currently Fully Taken
    IF @Status = 'Completed'
    BEGIN
        DECLARE @CurrentStatus NVARCHAR(30);
        SELECT @CurrentStatus = Status
        FROM    dbo.EventChecklistItems
        WHERE   ChecklistItemId = @ChecklistItemId;

        IF @CurrentStatus != 'Fully Taken'
        BEGIN
            SELECT -1 AS Result,
                   'Item must be Fully Taken before marking as Completed' AS
Message;
            RETURN;
        END
    END

    UPDATE dbo.EventChecklistItems SET
        ItemName           = ISNULL(@ItemName,           ItemName),
        QuantityNeeded     = ISNULL(@QuantityNeeded,    QuantityNeeded),
        QuantityUnit       = ISNULL(@QuantityUnit,      QuantityUnit),
        SuggestedMemberId  = ISNULL(@SuggestedMemberId,  SuggestedMemberId),
        IsMandatory        = ISNULL(@IsMandatory,       IsMandatory),
        AssignedMemberId   = ISNULL(@AssignedMemberId,   AssignedMemberId),
        Status             = ISNULL(@Status,            Status),
        UpdatedAt          = GETUTCDATE()
    WHERE ChecklistItemId = @ChecklistItemId;

    SELECT 1 AS Result, 'Updated successfully' AS Message;
END

```

3.3 sp_GetEventChecklist — ALTER

Returns all checklist items with claim aggregates and per-member context.

```

ALTER PROCEDURE dbo.sp_GetEventChecklist
    @EventId           BIGINT,
    @RequestingMemberId BIGINT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        ci.ChecklistItemId,
        ci.EventId,
        ci.ItemName,
        ci.QuantityNeeded,

```

```

        ci.QuantityUnit,
        ci.SuggestedMemberId,
        sm.DisplayName                                AS SuggestedMemberName,
        ci.IsMandatory,
        ci.AssignedMemberId,
        am.DisplayName                                AS AssignedMemberName,
        ci.Status,
        ci.CreatedAt,
        ISNULL(SUM(CASE WHEN cl.IsWithdrawn=0
                        THEN cl.QuantityClaimed END), 0) AS TotalQuantityClaimed,
        ci.QuantityNeeded - ISNULL(SUM(CASE WHEN cl.IsWithdrawn=0
                        THEN cl.QuantityClaimed END), 0) AS QuantityRemaining,
        MAX(CASE WHEN cl.MemberId=@RequestingMemberId AND cl.IsWithdrawn=0
                THEN 1 ELSE 0 END)                    AS IsClaimedByMe,
        ISNULL(MAX(CASE WHEN cl.MemberId=@RequestingMemberId AND cl.IsWithdrawn=0
                        THEN cl.QuantityClaimed END), 0) AS MyClaimedQuantity
FROM   dbo.EventChecklistItems ci
LEFT JOIN dbo.Members sm ON sm.MemberId = ci.SuggestedMemberId
LEFT JOIN dbo.Members am ON am.MemberId = ci.AssignedMemberId
LEFT JOIN dbo.EventChecklistClaims cl ON cl.ChecklistItemId = ci.ChecklistItemId
WHERE  ci.EventId = @EventId
GROUP BY
        ci.ChecklistItemId, ci.EventId, ci.ItemName, ci.QuantityNeeded,
        ci.QuantityUnit, ci.SuggestedMemberId, sm.DisplayName,
        ci.IsMandatory, ci.AssignedMemberId, am.DisplayName,
        ci.Status, ci.CreatedAt
ORDER BY ci.CreatedAt;
END

```

3.4 sp_GetEventChecklistClaims — CREATE (New)

Called once per event load alongside sp_GetEventChecklist. Returns all active claims for every item in the event. The service layer groups by ChecklistItemId to populate the Claims list on each EventChecklistItemResponse. One query — no N+1.

```

CREATE PROCEDURE dbo.sp_GetEventChecklistClaims
    @EventId BIGINT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        c.ClaimId,
        c.ChecklistItemId,
        c.MemberId,
        m.DisplayName AS MemberName,
        c.QuantityClaimed,
        c.ClaimedDate
    FROM   dbo.EventChecklistClaims      c
    JOIN   dbo.Members                  m ON m.MemberId      = c.MemberId
    JOIN   dbo.EventChecklistItems      ci ON ci.ChecklistItemId =
        c.ChecklistItemId
    WHERE  ci.EventId      = @EventId
          AND c.IsWithdrawn = 0
    ORDER BY c.ChecklistItemId, c.ClaimedDate;
END

-- Service usage pattern (C#):
-- 1. Call sp_GetEventChecklist      → List<ChecklistItemRow>
-- 2. Call sp_GetEventChecklistClaims → List<ClaimRow>
-- 3. Group ClaimRows by ChecklistItemId into a Dictionary
-- 4. Map to EventChecklistItemResponse, setting .Claims from the dictionary

```

3.5 sp_ClaimChecklistItem — CREATE (New)

```

CREATE PROCEDURE dbo.sp_ClaimChecklistItem
    @ChecklistItemId BIGINT,
    @MemberId        BIGINT,
    @QuantityClaimed INT = 1
AS BEGIN
    SET NOCOUNT ON;
    DECLARE @QuantityNeeded INT, @TotalClaimed INT;

    -- SERIALIZABLE + UPDLOCK prevents two concurrent claims from both
    -- reading the same "2 remaining" and both succeeding (race condition)
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    BEGIN TRANSACTION;

    BEGIN TRY
        -- Lock the row for the duration of this transaction
        SELECT @QuantityNeeded = QuantityNeeded
        FROM    dbo.EventChecklistItems WITH (UPDLOCK, ROWLOCK)
        WHERE   ChecklistItemId = @ChecklistItemId;

        SELECT @TotalClaimed = ISNULL(SUM(QuantityClaimed), 0)
        FROM    dbo.EventChecklistClaims
        WHERE   ChecklistItemId = @ChecklistItemId AND IsWithdrawn = 0;

        IF (@TotalClaimed + @QuantityClaimed) > @QuantityNeeded
        BEGIN
            ROLLBACK TRANSACTION;
            SELECT -1 AS Result, 'Quantity exceeds available' AS Message;
            RETURN;
        END

        IF EXISTS (SELECT 1 FROM dbo.EventChecklistClaims
                   WHERE ChecklistItemId=@ChecklistItemId AND MemberId=@MemberId)
        BEGIN
            UPDATE dbo.EventChecklistClaims SET
                IsWithdrawn=0, WithdrawnDate=NULL,
                QuantityClaimed=@QuantityClaimed, ClaimedDate=GETUTCDATE()
            WHERE ChecklistItemId=@ChecklistItemId AND MemberId=@MemberId;
        END
        ELSE
        BEGIN
            INSERT INTO dbo.EventChecklistClaims (ChecklistItemId, MemberId,
QuantityClaimed)
            VALUES (@ChecklistItemId, @MemberId, @QuantityClaimed);
        END

        DECLARE @NewTotal INT;
        SELECT @NewTotal = ISNULL(SUM(QuantityClaimed),0)
        FROM    dbo.EventChecklistClaims
        WHERE   ChecklistItemId=@ChecklistItemId AND IsWithdrawn=0;

        UPDATE dbo.EventChecklistItems SET
            Status = CASE WHEN @NewTotal=0                THEN 'Open'
                          WHEN @NewTotal>=@QuantityNeeded THEN 'Fully Taken'
                          ELSE 'Partially Taken' END,
            UpdatedAt = GETUTCDATE()
        WHERE ChecklistItemId = @ChecklistItemId;

        COMMIT TRANSACTION;
        SELECT 1 AS Result, 'Claimed successfully' AS Message;

    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        SELECT -1 AS Result, ERROR_MESSAGE() AS Message;
    END CATCH
END

```

```
END CATCH
END
```

3.6 sp_WithdrawChecklistClaim — CREATE (New)

```
CREATE PROCEDURE dbo.sp_WithdrawChecklistClaim
    @ChecklistItemId BIGINT,
    @MemberId        BIGINT
AS BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.EventChecklistClaims SET
        IsWithdrawn=1, WithdrawnDate=GETUTCDATE()
    WHERE ChecklistItemId=@ChecklistItemId
        AND MemberId=@MemberId AND IsWithdrawn=0;

    DECLARE @QuantityNeeded INT, @NewTotal INT;
    SELECT @QuantityNeeded=QuantityNeeded FROM dbo.EventChecklistItems
    WHERE ChecklistItemId=@ChecklistItemId;

    SELECT @NewTotal=ISNULL(SUM(QuantityClaimed),0)
    FROM    dbo.EventChecklistClaims
    WHERE   ChecklistItemId=@ChecklistItemId AND IsWithdrawn=0;

    UPDATE dbo.EventChecklistItems SET
        Status = CASE WHEN @NewTotal=0                THEN 'Open'
                      WHEN @NewTotal>=@QuantityNeeded THEN 'Fully Taken'
                      ELSE 'Partially Taken' END,
        UpdatedAt = GETUTCDATE()
    WHERE ChecklistItemId=@ChecklistItemId;
END
```

Section 4 — C# Models

Add to: FinTrackPortal.Models / GroupEvents / ChecklistModels.cs

4.1 AddChecklistItemRequest

```
public class AddChecklistItemRequest
{
    [Required][StringLength(300)]
    public string ItemName { get; set; }

    public int    QuantityNeeded { get; set; } = 1;
    public string? QuantityUnit  { get; set; }
    public long?  SuggestedMemberId { get; set; }

    // Advanced Mode - default off
    public bool  IsMandatory      { get; set; } = false;
    public long? AssignedMemberId { get; set; }
}
```

4.2 UpdateChecklistItemRequest

```
public class UpdateChecklistItemRequest
{

```

```

    public string? ItemName { get; set; }
    public int? QuantityNeeded { get; set; }
    public string? QuantityUnit { get; set; }
    public long? SuggestedMemberId { get; set; }
    public bool? IsMandatory { get; set; }
    public long? AssignedMemberId { get; set; }
    public string? Status { get; set; }
    // Status used by organizer to mark "Completed"
}

```

4.3 ClaimChecklistItemRequest

```

public class ClaimChecklistItemRequest
{
    public int QuantityClaimed { get; set; } = 1;
}

```

4.4 ChecklistClaimSummary

```

public class ChecklistClaimSummary
{
    public long MemberId { get; set; }
    public string MemberName { get; set; }
    public int QuantityClaimed { get; set; }
    public DateTime ClaimedDate { get; set; }
}

```

4.5 EventChecklistItemResponse

```

public class EventChecklistItemResponse
{
    public long ChecklistItemId { get; set; }
    public long EventId { get; set; }
    public string ItemName { get; set; }
    public int QuantityNeeded { get; set; }
    public string? QuantityUnit { get; set; }
    public int TotalQuantityClaimed { get; set; }
    public int QuantityRemaining { get; set; }
    public long? SuggestedMemberId { get; set; }
    public string? SuggestedMemberName { get; set; }
    public string Status { get; set; }
    // "Open" | "Partially Taken" | "Fully Taken" | "Completed"

    // Advanced Mode
    public bool IsMandatory { get; set; }
    public long? AssignedMemberId { get; set; }
    public string? AssignedMemberName { get; set; }
    public bool IsAssigned => AssignedMemberId.HasValue;
    // Computed – no DB column

    // Requesting member context
    public bool IsClaimedByMe { get; set; }
    public int MyClaimedQuantity { get; set; }

    public List<ChecklistClaimSummary> Claims { get; set; } = new();
    public DateTime CreatedAt { get; set; }
}

```

Section 5 — API Endpoints

Base path: /api/group/{groupId}/events/{eventId}/checklist

All require JWT Bearer. MemberId extracted from JWT claims — not passed in the URL.

RESPONSE WRAPPER — All endpoints return ApiResponse<T> — the standard wrapper already used across the backend. Shape: { "success": true, "message": "...", "data": <T>, "errors": null }. On failure: success=false, data=null, errors=[...]. Never return raw types directly.

Method	Path Suffix	Action	Notes
GET	(base path)	Get all checklist items	Returns ApiResponse<List<EventChecklistItemResponse>>. IsClaimedByMe from JWT.
POST	(base path)	Add item (organizer)	Body: AddChecklistItemRequest. Returns ApiResponse<EventChecklistItemResponse>. Audit: CHECKLIST_ITEM_CREATED
PUT	/ {itemId}	Update item (organizer)	Body: UpdateChecklistItemRequest. Returns ApiResponse<bool>. Audit: CHECKLIST_ITEM_UPDATED or _COMPLETED
DELETE	/ {itemId}	Delete item (organizer)	Returns ApiResponse<bool>. Cascades to Claims. Audit: CHECKLIST_ITEM_DELETED
POST	/ {itemId} /claim	Volunteer to bring item	Body: ClaimChecklistItemRequest. Returns ApiResponse<bool>. 400 if over-claiming.
DELETE	/ {itemId} /claim	Withdraw claim	Returns ApiResponse<bool>. MemberId from JWT. Status auto-recalculated.

Section 6 — Basic vs Advanced Mode

Mode is per item, driven by two flags. No EventType field. An event can mix Basic and Advanced items freely.

6.1 Basic Mode (Default — zero config needed)

```
Item: "Bring biryani — 4 packets"
  IsMandatory      = false    ← default
  AssignedMemberId = null     ← default
  IsAssigned       = false    ← computed
  Status           = "Open"   ← default

→ Any member taps "I'll take this", picks quantity
→ Status auto-updates: Open → Partially Taken → Fully Taken
→ Organizer marks Completed when confirmed at event
```

6.2 Advanced Mode (Organizer opts in per item)

```
Item: "Bring BBQ grill"
  IsMandatory      = true     ← organizer set
  AssignedMemberId = 1042     ← organizer set (Rajan)
  IsAssigned       = true     ← computed
```

```
→ Rajan sees: [🔴 Mandatory] [Assigned to you] [✓ I'll bring it]
→ Rajan claims via same POST .../claim endpoint
→ If QuantityNeeded > 1, others can also claim remaining quantity
→ Organizer can update/clear assignment at any time via PUT
```

6.3 Mixed Event Example

"Company Annual Picnic"

- Basic (IsMandatory=0, AssignedMemberId=null):
 - Extra chairs – qty 6
 - Sunscreen bottles – qty 4
 - Cricket bat / frisbee
- Advanced:
 - Sound system → IsMandatory=true, Assigned=Vikram
 - Company banner → IsMandatory=true, Assigned=Priya
 - Van booking → IsMandatory=true, Assigned=Organizer

Section 7 — Checklist Audit Logging

Use the existing audit infrastructure. Do NOT create a new table or stored procedure. The existing AuditLogs table, sp_WriteAuditLog, and IAuditLogRepository.WriteAsync() are already in place and registered in DI.

EXISTING SYSTEM — IAuditLogRepository is already registered in Program.cs. It is already injected in AuthController. Follow the same pattern for the Checklist controller.

7.1 New File — AuditChecklistActions.cs

Create this file in: FinTrackPortal.Common/

```
namespace FinTrackPortal.Common;

/// <summary>
/// Audit action constants for checklist and event attachment operations.
/// Mirrors the pattern in AuditAuthActions.cs.
/// </summary>
public static class AuditChecklistActions
{
    // Checklist item lifecycle
    public const string ChecklistItemCreated = "CHECKLIST_ITEM_CREATED";
    public const string ChecklistItemUpdated = "CHECKLIST_ITEM_UPDATED";
    public const string ChecklistItemCompleted = "CHECKLIST_ITEM_COMPLETED";
    public const string ChecklistItemDeleted = "CHECKLIST_ITEM_DELETED";

    // Claim lifecycle (recommended)
    public const string ChecklistClaimAdded = "CHECKLIST_CLAIM_ADDED";
    public const string ChecklistClaimWithdrawn = "CHECKLIST_CLAIM_WITHDRAWN";

    // Event attachments
    public const string EventAttachmentAdded = "EVENT_ATTACHMENT_ADDED";
    public const string EventAttachmentDeleted = "EVENT_ATTACHMENT_DELETED";
}
```

7.2 Inject IAuditLogRepository in the Controller

```
// In GroupEventsController (or dedicated ChecklistController):

private readonly IAuditLogRepository _audit;

public GroupEventsController(
    IGroupEventService events,
    IGroupChecklistService checklist,
    IAuditLogRepository audit,    // ← add this
    ...)
{
    _audit = audit;
    ...
}
```

7.3 Write Audit Calls — One Per Action

Call `_audit.WriteAsync()` immediately after each successful operation, before returning the `ApiResponse<T>`. Pass `ChecklistItemId` and `EventId` in the `Details` field.

```
// — POST (Add item) —————
var result = await _checklist.AddItemAsync(eventId, groupId, request);
await _audit.WriteAsync(
    memberId: memberId,
    action:    AuditChecklistActions.ChecklistItemCreated,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success:   true,
    details:   $"EventId={eventId}, ChecklistItemId={result.ChecklistItemId},
Item='{request.ItemName}'"
);
return Ok(new ApiResponse<EventChecklistItemResponse> { Success=true, Data=result
});

// — PUT (Update item) —————
// If Status is being set to "Completed", use ChecklistItemCompleted action
var auditAction = (request.Status == "Completed")
    ? AuditChecklistActions.ChecklistItemCompleted
    : AuditChecklistActions.ChecklistItemUpdated;

await _audit.WriteAsync(
    memberId: memberId,
    action:    auditAction,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success:   true,
    details:   $"EventId={eventId}, ChecklistItemId={itemId}"
);

// — DELETE (Delete item) —————
await _checklist.DeleteItemAsync(itemId, groupId);
await _audit.WriteAsync(
    memberId: memberId,
    action:    AuditChecklistActions.ChecklistItemDeleted,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success:   true,
    details:   $"EventId={eventId}, ChecklistItemId={itemId}"
);

// — POST .../claim —————
await _audit.WriteAsync(
```

```

        memberId: memberId,
        action: AuditChecklistActions.ChecklistClaimAdded,
        ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
        userAgent: Request.Headers["User-Agent"],
        success: true,
        details: $"ChecklistItemId={itemId}, EventId={eventId},
Qty={request.QuantityClaimed}"
    );

// — DELETE .../claim —————
await _audit.WriteAsync(
    memberId: memberId,
    action: AuditChecklistActions.ChecklistClaimWithdrawn,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success: true,
    details: $"ChecklistItemId={itemId}, EventId={eventId}"
);

// — Event Attachments —————
await _audit.WriteAsync(
    memberId: memberId,
    action: AuditChecklistActions.EventAttachmentAdded,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success: true,
    details: $"EventId={eventId}, AttachmentId={result.AttachmentId},
File='{request.FileName}'"
);

```

7.4 Audit Actions Reference Table

Constant	Action String	Triggered When	Details Field
ChecklistItemCreated	CHECKLIST_ITEM_CREATED	POST /checklist	EventId, ChecklistItemId, ItemName
ChecklistItemUpdated	CHECKLIST_ITEM_UPDATED	PUT /checklist/{id}	EventId, ChecklistItemId
ChecklistItemCompleted	CHECKLIST_ITEM_COMPLETED	PUT with Status=Completed	EventId, ChecklistItemId
ChecklistItemDeleted	CHECKLIST_ITEM_DELETED	DELETE /checklist/{id}	EventId, ChecklistItemId
EventAttachmentAdded	EVENT_ATTACHMENT_ADDED	POST /attachments	EventId, AttachmentId, FileName
EventAttachmentDeleted	EVENT_ATTACHMENT_DELETED	DELETE /attachments/{id}	EventId, AttachmentId

NOTE — Claim and Withdraw actions (POST/DELETE .../claim) are now also audited using CHECKLIST_CLAIM_ADDED and CHECKLIST_CLAIM_WITHDRAWN. The EventChecklistClaims table continues to store the full operational history. The audit log adds a security/compliance trail consistent with all other actions.

Section 8 — Complete Change Summary

Everything Abhilash needs to build. All changes relative to the original Phase 1 design.

#	Layer	Change	Type
1	Database	ALTER EventChecklistItems — DROP AssignedMemberId (Phase 1 forced column)	ALTER
2	Database	ALTER EventChecklistItems — ADD QuantityNeeded INT DEFAULT 1	ALTER
3	Database	ALTER EventChecklistItems — ADD QuantityUnit NVARCHAR(50) NULL	ALTER
4	Database	ALTER EventChecklistItems — ADD SuggestedMemberId BIGINT NULL FK	ALTER
5	Database	ALTER EventChecklistItems — ADD IsMandatory BIT DEFAULT 0	ALTER
6	Database	ALTER EventChecklistItems — ADD AssignedMemberId BIGINT NULL FK (Advanced mode)	ALTER
7	Database	CREATE dbo.EventChecklistClaims + performance index IX_EventChecklistClaims_ItemId_Withdrawn	CREATE
8	Stored Proc	sp_AddChecklistItem — ALTER: add all new parameters	ALTER
9	Stored Proc	sp_UpdateChecklistItem — ALTER: new fields + completion validation (must be Fully Taken)	ALTER
10	Stored Proc	sp_GetEventChecklist — ALTER: new columns, aggregates, member joins	ALTER
11	Stored Proc	sp_GetEventChecklistClaims — CREATE: all active claims for an event (no N+1)	CREATE
12	Stored Proc	sp_ClaimChecklistItem — CREATE: SERIALIZABLE transaction + UPDLOCK concurrency guard	CREATE
13	Stored Proc	sp_WithdrawChecklistClaim — CREATE: withdraw + status recalculation	CREATE
14	C# Model	AddChecklistItemRequest — add QuantityNeeded, QuantityUnit, SuggestedMemberId, IsMandatory, AssignedMemberId	UPDATE
15	C# Model	UpdateChecklistItemRequest — add same fields as nullable + Status for Completed	UPDATE
16	C# Model	ClaimChecklistItemRequest — new class (QuantityClaimed INT default 1)	CREATE
17	C# Model	ChecklistClaimSummary — new nested class for claims list in response	CREATE
18	C# Model	EventChecklistItemResponse — full replacement with all new fields + IsAssigned computed	UPDATE
19	API	GET /checklist — updated response (all new fields)	UPDATE
20	API	POST /checklist — updated request body (new fields all optional, backward compat)	UPDATE
21	API	PUT /checklist/{id} — 400 if Status=Completed and item not Fully Taken	UPDATE
22	API	DELETE /checklist/{id} — no change (cascade handles claims)	NONE
23	API	POST /checklist/{id}/claim — NEW endpoint (volunteer claim)	CREATE

24	API	DELETE /checklist/{id}/claim — NEW endpoint (withdraw claim)	CREATE
25	Audit	AuditChecklistActions.cs — NEW file (8 constants incl. CLAIM_ADDED, CLAIM_WITHDRAWN)	NEW
26	Audit	Inject IAuditLogRepository into controller; wire up all 8 audit calls	UPDATE
27	Audit	CHECKLIST_CLAIM_ADDED — fired on POST .../claim	UPDATE
28	Audit	CHECKLIST_CLAIM_WITHDRAWN — fired on DELETE .../claim	UPDATE

TOTAL (Checklist only) — 7 DB changes (1 DROP + 5 ADD + 1 CREATE table + 1 index) | 6 stored procedures (3 ALTER + 3 CREATE incl. GetEventChecklistClaims) | 5 C# models (3 UPDATE + 2 CREATE) | 6 API endpoints (3 UPDATE + 2 CREATE + 1 unchanged) | 1 new audit constants file (8 constants) + 8 audit wire-up calls. Concurrency: SERIALIZABLE + UPDLOCK on claim SP. Completion guard: Fully Taken required before Completed. Expense validation: ChecklistItemId cross-checked against EventId. See Section 9 for Checklist-Expense link changes. See Section 10 for Multi-Currency Expense Handling (1 new column: AmountOriginal).

Section 9 — Checklist-Expense Link

The checklist coordinates who buys what. The expense module handles settlement. These two systems remain separate and independent — we add only a lightweight reference link between them. No expense logic changes. No settlement logic changes. No new tables.

DESIGN DECISION — Add two nullable FK columns to the existing Expenses table. Both are optional — existing expense flows work exactly as before. The link is created only when a member taps "Add Expense" from a claimed checklist item.

9.1 DB Changes — ALTER dbo.Expenses

Add to the existing Expenses table. Both columns are nullable — backward compatible.

```
-- Tag an expense to a group event (e.g. farmhouse booking, biryani purchase)
ALTER TABLE dbo.Expenses
    ADD EventId BIGINT NULL
        REFERENCES dbo.GroupEvents(EventId);

-- Optionally link to the specific checklist item that triggered the purchase
ALTER TABLE dbo.Expenses
    ADD ChecklistItemId BIGINT NULL
        REFERENCES dbo.EventChecklistItems(ChecklistItemId);

-- Both NULL = normal expense (no event context)
-- EventId set, ChecklistItemId NULL = event expense (e.g. farmhouse booking)
-- Both set = expense linked to a specific checklist item (e.g. biryani)
```

9.2 ALTER sp_AddExpense

Add two optional parameters plus validation guard. All existing calls without these params continue to work.

```
ALTER PROCEDURE dbo.sp_AddExpense
```

```

-- ... existing parameters unchanged ...
@EventId          BIGINT = NULL,          -- NEW optional
@ChecklistItemId  BIGINT = NULL          -- NEW optional
AS BEGIN
    SET NOCOUNT ON;

    -- Expense link validation:
    -- If ChecklistItemId is supplied, verify it belongs to the same EventId.
    -- Prevents cross-event data corruption.
    IF @ChecklistItemId IS NOT NULL AND @EventId IS NOT NULL
    BEGIN
        IF NOT EXISTS (
            SELECT 1 FROM dbo.EventChecklistItems
            WHERE ChecklistItemId = @ChecklistItemId
              AND EventId       = @EventId
        )
        BEGIN
            SELECT -1 AS Result,
                   'ChecklistItemId does not belong to the specified EventId' AS
Message;
            RETURN;
        END
    END

    -- ... existing INSERT logic unchanged ...
    -- Add to the INSERT statement:
    INSERT INTO dbo.Expenses (... , EventId, ChecklistItemId)
    VALUES (... , @EventId, @ChecklistItemId);
END

```

NOTE — `sp_AddPersonalExpense` does NOT get `ChecklistItemId` — personal expenses are individual and not linked to group checklist items. It may receive `EventId` if the personal expense is contextually related to an event (e.g. travel cost), but this is optional.

9.3 CREATE `sp_GetEventExpenses` — New

Called by the Event Expenses summary tab. Returns all expenses tagged to a specific event.

```

CREATE PROCEDURE dbo.sp_GetEventExpenses
    @EventId BIGINT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        e.ExpenseId,
        e.GroupId,
        e.Description,
        e.Amount,
        e.CurrencyCode,
        e.PaidByMemberId,
        m.DisplayName          AS PaidByMemberName,
        e.EventId,
        e.ChecklistItemId,
        ci.ItemName            AS ChecklistItemName,
        e.CreatedAt
    FROM   dbo.Expenses e
    JOIN   dbo.Members m ON m.MemberId = e.PaidByMemberId
    LEFT JOIN dbo.EventChecklistItems ci ON ci.ChecklistItemId = e.ChecklistItemId
    WHERE  e.EventId = @EventId
    ORDER BY e.CreatedAt DESC;
END

```

9.4 C# Model Changes

Add to the existing expense request and response models in ExpenseModels.cs:

```
// AddExpenseRequest - add two optional fields
public class AddExpenseRequest
{
    // ... existing fields unchanged ...

    // Optional context links - both null by default
    public long? EventId      { get; set; }
    public long? ChecklistItemId { get; set; }
}

// ExpenseResponse - add linked context fields
public class ExpenseResponse
{
    // ... existing fields unchanged ...

    public long?      EventId      { get; set; }
    public long?      ChecklistItemId { get; set; }
    public string?    ChecklistItemName { get; set; }
}

// EventExpenseSummaryResponse - new, for Event Expenses tab
public class EventExpenseSummaryResponse
{
    public long      ExpenseId      { get; set; }
    public string    Description    { get; set; }
    public decimal   Amount         { get; set; }
    public string    CurrencyCode   { get; set; }
    public long      PaidByMemberId { get; set; }
    public string    PaidByMemberName { get; set; }
    public long?     ChecklistItemId { get; set; }
    public string?   ChecklistItemName { get; set; }
    public DateTime  CreatedAt      { get; set; }
}
```

9.5 New API Endpoint — GET Event Expenses

```
// Add to GroupEventsController (or ExpenseController):

// GET /api/group/{groupId}/events/{eventId}/expenses
// Returns all expenses tagged to this event
// Response: ApiResponse<List<EventExpenseSummaryResponse>>

[HttpGet("~/api/group/{groupId}/events/{eventId}/expenses")]
public async Task<IActionResult> GetEventExpenses(
    long groupId, long eventId)
{
    var expenses = await _expenseService.GetEventExpensesAsync(eventId, groupId);
    return Ok(new ApiResponse<List<EventExpenseSummaryResponse>>
    {
        Success = true,
        Data     = expenses
    });
}
```

9.6 Existing POST /expenses — No Route Change

IMPORTANT — The existing POST /api/group/{gId}/expenses endpoint does NOT change its URL or behaviour. Manus simply passes the optional EventId and ChecklistItemId fields when the expense is being added from the event/checklist context. When added from the normal expenses screen, these fields are not sent — and everything works as before.

9.7 How the Link is Created — Flow Summary

Scenario	EventId sent?	ChecklistItemId sent?
Normal group expense (no event context)	null	null
Expense added from Event screen (e.g. farmhouse)	event ID	null
Expense added via "Add Expense" on a claimed item	event ID	item ID
Personal expense	null / opt	null (never)

9.8 Audit Logging — Add to AuditChecklistActions.cs

Add these two constants to the existing AuditChecklistActions class:

```
// Add to AuditChecklistActions.cs
public const string ChecklistExpenseLinked = "CHECKLIST_EXPENSE_LINKED";
// Fired when an expense is added with a ChecklistItemId set

// Write audit in ExpenseController when ChecklistItemId != null:
await _audit.WriteAsync(
    memberId: memberId,
    action:    AuditChecklistActions.ChecklistExpenseLinked,
    ipAddress: HttpContext.Connection.RemoteIpAddress?.ToString(),
    userAgent: Request.Headers["User-Agent"],
    success:   true,
    details:   $"ExpenseId={expenseId}, ChecklistItemId={request.ChecklistItemId},
    EventId={request.EventId}"
);
```

9.9 Complete Change Summary — Section 9 Only

#	Layer	Change	Type
1	Database	ALTER Expenses — ADD EventId BIGINT NULL FK → GroupEvents	ALTER
2	Database	ALTER Expenses — ADD ChecklistItemId BIGINT NULL FK → EventChecklistItems	ALTER
3	Stored Proc	ALTER sp_AddExpense — ADD @EventId NULL, @ChecklistItemId NULL params	ALTER
4	Stored Proc	CREATE sp_GetEventExpenses — returns expenses tagged to a specific event	CREATE
5	C# Model	AddExpenseRequest — ADD EventId?, ChecklistItemId? (both optional)	UPDATE
6	C# Model	ExpenseResponse — ADD EventId?, ChecklistItemId?, ChecklistItemName?	UPDATE
7	C# Model	EventExpenseSummaryResponse — NEW class for Event Expenses tab	NEW

8	API	GET /api/group/{gId}/events/{eId}/expenses — NEW endpoint	NEW
9	API	POST /api/group/{gId}/expenses — NO URL change; accepts 2 new optional fields	ALTER
10	Audit	AuditChecklistActions: ADD ChecklistExpenseLinked constant	UPDATE

SECTION 9 TOTAL — 2 DB column additions to Expenses | 1 SP altered + 1 SP new | 2 models updated + 1 model new | 1 API endpoint new + 1 existing endpoint updated (backward compat). Zero new tables. Zero changes to split/settlement logic.

Section 10 — Multi-Currency Expense Handling

Members may pay for expenses in a foreign currency (e.g. Omani Riyal on an Oman trip, Euro in Albania). This section defines the minimal backend change needed to preserve the original payment amount for transparency and audit — without adding complexity to settlement logic.

DESIGN DECISION — Store AmountOriginal alongside the existing Amount (AED settlement value). This is a single nullable column addition — minimal change, maximum audit clarity. ExchangeRate is NOT stored. Settlement uses Amount only. AmountOriginal is reference data — it never participates in any calculation.

10.1 Why Store AmountOriginal?

Without AmountOriginal	With AmountOriginal
Expense shows: 97 AED — member has no way to verify the entry matches what they paid	Expense shows: 10 OMR → 97 AED — member can confirm 10 OMR is correct at a glance
If a dispute arises, there is no record of the original foreign amount	Full audit trail: what was paid, in what currency, declared as what AED value
Groups with expat members or frequent travel cannot verify historical expenses	Original amount preserved — even after viewing months later

10.2 Data Flow Summary

Field	Stored in DB?	Used in settlement?
Amount	YES — existing column	YES — only this value
CurrencyCode	YES — existing column	NO — reference only
AmountOriginal	YES — new column (nullable)	NO — audit/reference only
ExchangeRate	NO — never stored	NO — UI display only

10.3 DB Change — ALTER dbo.Expenses

One new nullable column. All existing rows default to NULL. Fully backward compatible.

```
-- Original amount in the currency the member actually paid in.
```

```
-- NULL when CurrencyCode is NULL or matches the group default (AED).
-- NEVER used in settlement – stored for transparency and audit only.
ALTER TABLE dbo.Expenses
    ADD AmountOriginal DECIMAL(18,2) NULL;

-- ExchangeRate: intentionally NOT added.
-- Rate is derived in UI as AmountOriginal / Amount – display only.
```

10.4 Expenses Column Reference — After Change

```
dbo.Expenses (currency-relevant columns)
-----
Amount          DECIMAL(18,2) NOT NULL  – settlement amount in group default
currency (AED)                                     ALL calculations use this value
AmountOriginal  DECIMAL(18,2) NULL      – NEW: original foreign currency amount
                                         e.g. 10.00 for "10 OMR". NULL if same
currency.                                         Reference/audit only. Never used in
math.
CurrencyCode    NVARCHAR(10)  NULL      – foreign currency code (e.g. 'OMR', 'EUR')
                                         NULL = group default (AED)

-- ExchangeRate: NOT stored. Derived in UI as Amount / AmountOriginal.
```

SETTLEMENT RULE (unchanged) — All split, balance, and settlement calculations use Amount only. AmountOriginal is stored but is never read by any calculation stored procedure. No changes to sp_GetGroupBalances, sp_SettleExpense, or any split logic.

10.5 ALTER sp_AddExpense — One New Optional Parameter

Add one optional parameter. All existing calls continue to work without sending it.

```
ALTER PROCEDURE dbo.sp_AddExpense
    -- ... all existing parameters unchanged ...
    @AmountOriginal DECIMAL(18,2) = NULL -- NEW: original foreign amount (e.g.
10 OMR)
AS BEGIN
    SET NOCOUNT ON;

    -- ... existing validation unchanged (EventId/ChecklistItemId cross-check from
Section 9) ...

    -- Rule 1: AmountOriginal is MANDATORY when CurrencyCode is provided.
    -- If the member paid in a foreign currency, the original amount must be
recorded.
    IF (@CurrencyCode IS NOT NULL AND @CurrencyCode <> '') AND @AmountOriginal IS
NULL
    BEGIN
        SELECT -1 AS Result, 'AmountOriginal is required when CurrencyCode is
provided' AS Message;
        RETURN;
    END

    -- Rule 2: AmountOriginal is only meaningful alongside a CurrencyCode.
    IF @AmountOriginal IS NOT NULL AND (@CurrencyCode IS NULL OR @CurrencyCode = '')
    BEGIN
        SELECT -1 AS Result, 'CurrencyCode is required when AmountOriginal is
provided' AS Message;
        RETURN;
    END
```

```

END

-- Rule 3: AmountOriginal must be positive if provided.
IF @AmountOriginal IS NOT NULL AND @AmountOriginal <= 0
BEGIN
    SELECT -1 AS Result, 'AmountOriginal must be greater than zero' AS Message;
    RETURN;
END

-- Add to the existing INSERT:
INSERT INTO dbo.Expenses (... , AmountOriginal)
VALUES (... , @AmountOriginal);
END

```

10.6 ALTER sp_GetEventExpenses — Return AmountOriginal

Update the SELECT in sp_GetEventExpenses (introduced in Section 9.3) to include the new column:

```

ALTER PROCEDURE dbo.sp_GetEventExpenses
    @EventId BIGINT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        e.ExpenseId,
        e.GroupId,
        e.Description,
        e.Amount,                                -- settlement amount (AED)
        e.AmountOriginal,                        -- NEW: original foreign amount (nullable)
        e.CurrencyCode,
        e.PaidByMemberId,
        m.DisplayName AS PaidByMemberName,
        e.EventId,
        e.ChecklistItemId,
        ci.ItemName AS ChecklistItemName,
        e.CreatedAt
    FROM   dbo.Expenses e
    JOIN   dbo.Members m ON m.MemberId = e.PaidByMemberId
    LEFT JOIN dbo.EventChecklistItems ci ON ci.ChecklistItemId = e.ChecklistItemId
    WHERE  e.EventId = @EventId
    ORDER BY e.CreatedAt DESC;
END

```

10.7 C# Model Changes

Add AmountOriginal to the two expense models updated in Section 9.4:

```

// AddExpenseRequest — one new optional field
public class AddExpenseRequest
{
    // ... existing fields unchanged ...
    public long?   EventId      { get; set; }    // from Section 9
    public long?   ChecklistItemId { get; set; }  // from Section 9

    // NEW — optional; only sent for foreign-currency expenses
    public decimal? AmountOriginal { get; set; } // e.g. 10 OMR
    // CurrencyCode already exists — no change
}

// EventExpenseSummaryResponse — one new field
public class EventExpenseSummaryResponse
{
    public long    ExpenseId      { get; set; }
    public string  Description    { get; set; }
}

```

```

    public decimal Amount { get; set; } // settlement amount (AED)
    public decimal? AmountOriginal { get; set; } // NEW: original foreign
    amount
    public string? CurrencyCode { get; set; }
    public long PaidByMemberId { get; set; }
    public string PaidByMemberName { get; set; }
    public long? ChecklistItemId { get; set; }
    public string? ChecklistItemName { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

10.8 What Is Explicitly NOT Changing

Item	Changed?	Note
ExchangeRate column in DB	NO X	UI-only derived value. Never stored.
Settlement / split stored procs	NO X	Amount is already the AED value. No logic changes.
PaymentMethod column	NO X	Out of scope for this requirement.
CurrencyWallets table	NO X	Separate feature — future scope.
External FX API	NO X	No live rates. Member declares AED equivalent.
New API endpoints	NO X	Existing POST /expenses handles the new optional field.
Group balance / settle endpoints	NO X	Use Amount only — unchanged.

10.9 Complete Change Summary — Section 10 Only

#	Layer	Change	Type
1	Database	ALTER Expenses — ADD AmountOriginal DECIMAL(18,2) NULL (foreign currency reference)	ALTER
2	Stored Proc	ALTER sp_AddExpense — ADD @AmountOriginal DECIMAL NULL + 2 validation guards	ALTER
3	Stored Proc	ALTER sp_GetEventExpenses — ADD AmountOriginal to SELECT list	ALTER
4	C# Model	AddExpenseRequest — ADD AmountOriginal? (optional, foreign currency only)	UPDATE
5	C# Model	EventExpenseSummaryResponse — ADD AmountOriginal? nullable field	UPDATE

SECTION 10 TOTAL — 1 DB column addition (AmountOriginal on Expenses) | 2 stored procedures altered (sp_AddExpense + sp_GetEventExpenses) | 2 C# models updated (AddExpenseRequest + EventExpenseSummaryResponse). Zero new tables. Zero new stored procedures. Zero new API endpoints. Zero changes to settlement, split, or balance logic. ExchangeRate is NEVER stored — it is a UI-only derived value.

Section 11 — In-App Notifications

The notification system already exists in the codebase. This section documents the existing implementation and identifies exactly which new triggers need to be wired in for the checklist and event modules. Do NOT recreate any part of the existing system.

IMPORTANT — DO NOT RECREATE — The Notifications table, all stored procedures, INotificationRepository, INotificationService, NotificationService, NotificationController, and DI registrations already exist and are working. The expense trigger is already wired. Only the checklist and event triggers need to be added.

11.1 Existing Table — dbo.Notifications (already in production)

Do NOT run CREATE TABLE. This table already exists. The actual column names differ from the initial design spec:

```
dbo.Notifications -- EXISTING TABLE (do not recreate)

NotificationId  BIGINT          IDENTITY PK
MemberId        BIGINT          NOT NULL  FK → dbo.Members (MemberId)
Title           NVARCHAR(200)  NOT NULL
Body            NVARCHAR(2000) NULL      ← field is "Body" not "Message"
NotificationType NVARCHAR(50)    NULL      ← field is "NotificationType" not "Type"
IsRead          BIT          NOT NULL  DEFAULT 0
LinkUrl         NVARCHAR(500) NULL      ← extra field for deep-link navigation
CreatedDate     DATETIME2   NOT NULL  DEFAULT GETUTCDATE() ← "CreatedDate" not
"CreatedAt"
ModifiedDate    DATETIME2   NULL
IsActive        BIT          NOT NULL  DEFAULT 1          ← soft-delete flag

-- Use these exact column names in any new SP or query.
```

11.2 Existing Stored Procedures (already exist — do not recreate)

SP Name	Parameters	Purpose
sp_CreateNotification	@MemberId, @Title, @Body, @NotificationType, @LinkUrl	Insert one notification row. Returns new NotificationId.
sp_GetNotifications	@MemberId, @UnreadOnly BIT=0, @Take INT=100	Return latest N notifications for a member.
sp_MarkNotificationRead	@NotificationId, @MemberId	Mark one notification read. Ownership-checked. Returns RowsUpdated.
sp_MarkAllNotificationsRead	@MemberId	Mark all unread notifications for a member as read.

11.3 Existing Service Interface (already exists — do not recreate)

```
// INotificationService – already in FinTrackPortal.Services
public interface INotificationService
{
    Task<IReadOnlyList<NotificationItem>> GetAsync(long memberId, bool unreadOnly,
int take);
    Task<bool> MarkReadAsync(long notificationId, long memberId);
    Task MarkAllReadAsync(long memberId);
}
```

```
// This is the method to call when wiring new triggers:
Task<long> CreateAsync(
    long memberId,           // one recipient per call
    string title,
    string? body,            // ← parameter is "body" not "message"
    string? notificationType, // ← "notificationType" not "type"
    string? linkUrl);        // ← deep-link for frontend navigation
}
```

11.4 Existing NotificationController (already exists — do not recreate)

```
// Route: /api/Notification (capital N — ASP.NET convention from [controller])

GET /api/Notification // ?unreadOnly=false&take=100
PUT /api/Notification/{notificationId}/read
PUT /api/Notification/read-all

// All return: ApiResponse<T>
// All require: Authorization: Bearer <token>
// MemberId is extracted from JWT — not passed as a body param
```

11.5 Already Working Trigger — Expense Added (reference pattern)

ExpenseController.Add() already implements the notification trigger. Use this as the reference pattern for all new triggers:

```
// ExpenseController.Add() — existing implementation (do not change)
// After successful expense insert:

var members = await _groupService.GetGroupMembersAsync(request.GroupId);
if (members.IsSuccess && members.Data != null)
{
    foreach (var member in members.Data.Where(m => m.MemberId != memberId))
    {
        await _notifications.CreateAsync(
            member.MemberId,
            "New Expense Added",
            $"{createdBy} added '{request.Description}' - {request.Amount:N2}
{request.CurrencyCode}",
            "expense",
            $" /group/{request.GroupId}");
    }
}

// Key points from this pattern:
// 1. Get members via _groupService.GetGroupMembersAsync(groupId)
// 2. Exclude actor: .Where(m => m.MemberId != memberId)
// 3. Call _notifications.CreateAsync() per recipient — one row each
// 4. Fire after successful DB operation only
// 5. linkUrl is a frontend deep-link string — use for navigation
```

11.6 New Triggers Required — 5 to Add

These 5 triggers do not exist yet. Wire them in the new ChecklistController and the existing GroupEventService using the same pattern as ExpenseController.Add().

Where to add	After operation	Title	Body template	Exclude
ChecklistController AddChecklistItem	After item INSERT succeeds	"New Checklist Item"	"New item added to checklist: {itemName}"	Organizer

ChecklistController ClaimItem	After claim INSERT succeeds	"Checklist Item Claimed"	"{memberName} claimed: {itemName} ({qty} {unit})"	Claimer
ChecklistController WithdrawClaim	After IsWithdrawn=1 update	"Claim Removed"	"{memberName} removed their claim for {itemName}"	Actor
GroupEventsController CreateEvent	After event INSERT succeeds	"New Event Created"	"New event: {title} – {eventDate}"	Creator
GroupEventsController UpdateEvent	ONLY if Date or Location changed	"Event Updated"	"Event updated: {title}"	Updater

UpdateEvent rule — Only fire the notification when EventDate or Location actually changes. Capture the existing values before the UPDATE. Compare: if neither changed, skip the notification entirely. Do NOT notify for title or description-only edits.

11.7 How to Wire — Copy the Expense Pattern

The injection and loop pattern is already established. Follow it exactly for the new triggers:

```
// STEP 1 — Inject both IGroupService and INotificationService in the constructor
// (GroupService gives you GetGroupMembersAsync; already injected in
ExpenseController)

// STEP 2 — After a successful operation, add this block:
var members = await _groupService.GetGroupMembersAsync(groupId);
if (members.IsSuccess && members.Data != null)
{
    foreach (var member in members.Data.Where(m => m.MemberId != actorMemberId))
    {
        await _notifications.CreateAsync(
            member.MemberId,
            "<Title from table above>",
            $"<Body template from table above>",
            "<notificationType>", // "checklist" | "event"
            $"<linkUrl>");        // e.g. "/group/{groupId}/events/{eventId}"
    }
}

// STEP 3 — The notification call is fire-and-forget (no await on failure handling).
// A notification failure must NEVER cause the primary operation to fail.
// Wrap in try/catch if needed to ensure silent failure:
try { /* notification loop */ }
catch (Exception ex) { _logger.LogWarning(ex, "Notification delivery failed"); }
```

11.8 notificationType Values — Use Lowercase to Match Existing Pattern

ExpenseController uses "expense" (lowercase). Use these values consistently for the new triggers:

Trigger	notificationType value	linkUrl example
Expense Added	"expense"	"/group/{groupId}"
Checklist Item Added	"checklist"	"/group/{groupId}/events/{eventId}"
Checklist Item Claimed	"checklist"	"/group/{groupId}/events/{eventId}"
Checklist Claim Removed	"checklist"	"/group/{groupId}/events/{eventId}"

Event Created	"event"	"/group/{groupId}/events/{eventId}"
Event Updated	"event"	"/group/{groupId}/events/{eventId}"

11.9 What Is Already Done — Status Checklist

Item	Status	Note
dbo.Notifications table	✓ Done	Exists in production
sp_CreateNotification	✓ Done	Params: Body, NotificationType, LinkUrl
sp_GetNotifications	✓ Done	Supports unreadOnly + take filters
sp_MarkNotificationRead	✓ Done	Ownership-checked
sp_MarkAllNotificationsRead	✓ Done	Bulk mark for a member
INotificationRepository + implementation	✓ Done	All 4 methods implemented
INotificationService + implementation	✓ Done	Thin wrapper over repository
NotificationController + 3 endpoints	✓ Done	GET + PUT read + PUT read-all
DI registration in Program.cs	✓ Done	Both repo and service registered
Expense Added trigger	✓ Done	In ExpenseController.Add()
Member Added to Group trigger	✓ Done	In GroupController.AddMember()
Checklist Item Added trigger	✗ Missing	Add to new ChecklistController
Checklist Item Claimed trigger	✗ Missing	Add to new ChecklistController
Checklist Claim Withdrawn trigger	✗ Missing	Add to new ChecklistController
Event Created trigger	✗ Missing	Add to GroupEventsController
Event Updated trigger (date/loc only)	✗ Missing	Add to GroupEventsController

11.10 Change Summary — Section 11 (New Work Only)

Everything already in production is excluded. Only the 5 missing triggers need to be added:

#	Layer	Change	Type
1	Controller	ChecklistController.AddChecklistItem — add notification loop after item INSERT	UPDATE
2	Controller	ChecklistController.ClaimItem — add notification loop after claim INSERT	UPDATE
3	Controller	ChecklistController.WithdrawClaim — add notification loop after withdraw	UPDATE
4	Controller	GroupEventsController.CreateEvent — add notification loop after event INSERT	UPDATE
5	Controller	GroupEventsController.UpdateEvent — add conditional loop IF Date or Location changed	UPDATE

SECTION 11 TOTAL (new work) — 0 new tables | 0 new SPs | 0 new models | 0 new API endpoints. Only 5 controller-level notification loops to add: 3 in ChecklistController + 2 in GroupEventsController. All existing notification infrastructure (table, SPs, service, controller, DI) is already in place.